

Replit Website — Cart, Order & Payment Flow Prompt

Copy the following prompt into Replit. The goal is to fix the cart flow and implement the order/payment workflow exactly as described.

IMPORTANT: Fix and implement the complete e-commerce order flow.

The current **Add to Cart** functionality is not working correctly. Please inspect the existing code and fix it first. Do not create a separate unrelated system. Keep the current website design and existing product/admin structure wherever possible.

1. CUSTOMER — ADD TO CART

When a customer clicks **Add to Cart**, the selected product must actually be added to the customer's cart. The cart should show the product name, image, quantity, unit price, subtotal, and total product amount. Customers must be able to increase/decrease quantity and remove products.

2. CHECKOUT / PLACE ORDER — NO IMMEDIATE PAYMENT

The customer should NOT be charged immediately when placing the order. First, the customer submits the order/request. At this stage, save the order in the database and make it visible in the Admin Panel.

The order should have a clear status such as **Order Received / Awaiting Admin Confirmation**.

Save at minimum: order ID, customer details, delivery address, products, quantities, product subtotal, order date/time, and order status.

3. ADMIN PANEL — REVIEW ORDER

The Admin Panel must show all newly submitted customer orders. Admin should be able to open an order and see all order details and customer information.

The admin must be able to set/edit the **Delivery Charge** for that specific order from the Admin Panel.

The system should then calculate:

Product Subtotal + Delivery Charge = Final Amount Payable

Do not hard-code the delivery charge. It must be controlled by the admin for each order.

4. ADMIN SENDS PAYMENT REQUEST

After reviewing the order and setting the delivery charge, the admin can mark the order as **Payment Requested / Awaiting Customer Payment**.

The customer should then see the final amount, including delivery charge, in their order page. Clearly show that the amount has been confirmed by the admin and that payment is now required.

5. CUSTOMER MAKES PAYMENT

Only after the admin has confirmed the final amount should the customer proceed with payment.

After payment, update the order status to something like **Payment Received / Order Confirmed**. The admin must also be able to see the payment status in the Admin Panel.

If the existing project already has a payment gateway, integrate this flow with it instead of replacing it

unnecessarily. If payment integration is not currently available, structure the order system so the payment step can be connected cleanly without breaking the order workflow.

6. DELIVERY ESTIMATE

Once the order is confirmed/paid, show the customer an estimated delivery time of **1–2 days**.

Display this clearly on the order tracking/status page, for example: **Estimated Delivery: 1–2 Days**.

7. ORDER STATUS / TRACKING

The customer should be able to open their order and see its current status. Use a simple status flow such as:

Order Received → Awaiting Admin Confirmation → Payment Requested → Payment Received / Order Confirmed → Preparing Order → Out for Delivery → Delivered

The Admin Panel should allow the admin to update the order status. When the admin changes the status, the customer should see the updated status on their order page.

For example, when the admin changes the order to **Out for Delivery**, the customer should clearly see **Out for Delivery**. When the admin changes it to **Delivered**, the customer should see **Delivered**.

8. IMPORTANT DATA/SECURITY REQUIREMENTS

Make sure each order belongs to the correct customer and customers can only see their own orders.

Admin-only controls such as changing delivery charges, payment-request status, and order status must not be accessible to normal customers.

Do not expose sensitive payment information. Do not mark an order as paid merely because the customer opened the payment page. Payment status must only become successful after the payment flow/gateway confirms it, or through the project's intended verified payment mechanism.

9. UI REQUIREMENTS

Keep the website's existing design, colors, typography, and layout unless a change is necessary for this workflow. Make the customer experience simple and clear. Add clear buttons and status labels such as **Add to Cart**, **Place Order**, **Pay Now**, **Awaiting Payment**, **Out for Delivery**, and **Delivered**.

10. IMPLEMENTATION REQUIREMENT

Before changing the code, inspect the current cart, product, authentication, database, admin panel, and payment-related code. Find why Add to Cart is currently failing and fix the root cause instead of applying a temporary UI-only fix.

Then implement the complete flow end-to-end and test it using a sample customer account and admin account.

EXPECTED FINAL FLOW:

Customer selects product → Add to Cart → Cart works → Customer places order → Order is saved in database → Order appears in Admin Panel → Admin reviews order → Admin sets delivery charge → Admin sends payment request → Customer sees final amount → Customer pays → Payment is verified → Order becomes confirmed → Customer sees estimated delivery of 1–2 days → Admin updates status → Customer can track status until Out for Delivery → Delivered.

Please implement this carefully across frontend, backend, database, authentication/authorization, and payment flow. Make sure existing functionality is not unnecessarily broken. After implementation, test the

entire flow and fix any errors you find.

Note: The key idea is that payment happens only after the admin reviews the order and sets the delivery charge. The order is created first, then the admin confirms the final payable amount, and only then does the customer pay.